

Project Code Export

Includes source code and frontend screenshots

README.md

```
# Intelligent Insurance Agent
```

This project is a starter RAG-style insurance assistant with:

- a FastAPI service for chat, health, upload, and document listing
- a simple CLI RAG workflow for indexing documents and answering prompts
- sample insurance-related data and prompts

```
## Project structure
```

- src/intelligent_insurance_agent/Api: FastAPI endpoint modules
- src/intelligent_insurance_agent/Core: environment config and testing prompts
- src/intelligent_insurance_agent/Database: sample insurance data and seeding helpers
- src/intelligent_insurance_agent/Model: chat and document abstractions
- src/intelligent_insurance_agent/RAG: loader, splitter, embedding, vector store, retriever, and chain components
- src/intelligent_insurance_agent/main.py: RAG CLI implementation

```
## Setup
```

```
```bash
cd /home/labuser/project/intelligent_insurance_agent
PYTHONPATH=src python -m pip install -e .
```
```

```
## Run tests
```

```
```bash
PYTHONPATH=src python -m pytest
```
```

```
## Run the RAG CLI
```

```
```bash
PYTHONPATH=src python main.py --document /path/to/document.txt --prompt "What does the policy cover?"
```
```

Or run interactively:

```
```bash
PYTHONPATH=src python main.py
```
```

```
## Run the API
```

```
```bash
PYTHONPATH=src uvicorn intelligent_insurance_agent.api:app --host 127.0.0.1 --port 8000
```
```

Then visit:

- <http://127.0.0.1:8000/health>
- <http://127.0.0.1:8000/docs>
- <http://127.0.0.1:8000/> for the chatbot frontend

main.py

```
"""Top-level entrypoint for the insurance RAG application."""
from intelligent_insurance_agent.main import main
```

```
if __name__ == "__main__":
    main()
```

pyproject.toml

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "intelligent-insurance-agent"
version = "0.1.0"
description = "Starter project for an intelligent insurance assistant"
requires-python = ">=3.10"
dependencies = []

[project.scripts]
insurance-agent = "intelligent_insurance_agent.agent:main"
insurance-api = "intelligent_insurance_agent.api:main"
insurance-rag = "intelligent_insurance_agent.main:main"

[tool.pytest.ini_options]
testpaths = ["tests"]

[tool.setuptools.packages.find]
where = ["src"]
```

src/intelligent_insurance_agent/Api/__init__.py

```
from __future__ import annotations

import os
from pathlib import Path

UPLOAD_DIR = Path(os.getenv("INSURANCE_UPLOAD_DIR", "./uploads"))
UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
```

src/intelligent_insurance_agent/Api/chat.py

```
from __future__ import annotations

from fastapi import APIRouter
from pydantic import BaseModel

from ..agent import generate_response

router = APIRouter()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

@router.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest) -> ChatResponse:
    reply = generate_response(req.message)
    return ChatResponse(reply=reply)
```

src/intelligent_insurance_agent/Api/documents.py

```

from __future__ import annotations

from typing import List

from fastapi import APIRouter
from pydantic import BaseModel

router = APIRouter()

class DocumentSummary(BaseModel):
    filename: str
    size: int
    path: str

@router.get("/documents", response_model=List[DocumentSummary])
def documents() -> List[DocumentSummary]:
    from .. import api

    files = []
    for path in sorted(api.UPLOAD_DIR.iterdir()):
        if path.is_file():
            files.append(
                DocumentSummary(
                    filename=path.name,
                    size=path.stat().st_size,
                    path=str(path.resolve()),
                )
            )
    return files

```

src/intelligent_insurance_agent/Api/health.py

```

from __future__ import annotations

from typing import Dict

from fastapi import APIRouter

router = APIRouter()

@router.get("/health")
def health() -> Dict[str, str]:
    return {"status": "ok"}

```

src/intelligent_insurance_agent/Api/rag.py

```

"""RAG chat endpoint using the shared InsuranceRAGApp."""

from __future__ import annotations

from pathlib import Path

from fastapi import APIRouter
from pydantic import BaseModel

from ..main import InsuranceRAGApp

router = APIRouter()
rag_app = InsuranceRAGApp()

class RAGChatRequest(BaseModel):
    prompt: str

```

```

class RAGChatResponse(BaseModel):
    response: str

@router.post("/rag-chat", response_model=RAGChatResponse)
def rag_chat(req: RAGChatRequest) -> RAGChatResponse:
    response = rag_app.answer_query(req.prompt)
    return RAGChatResponse(response=response)

def ingest_existing_documents(upload_dir: Path) -> int:
    file_paths = [path for path in upload_dir.iterdir() if path.is_file()]
    return rag_app.ingest_documents(file_paths)

```

src/intelligent_insurance_agent/Api/upload.py

```

from __future__ import annotations

from pathlib import Path

from fastapi import APIRouter, File, HTTPException, UploadFile
from pydantic import BaseModel

from .rag import rag_app

router = APIRouter()

class DocumentSummary(BaseModel):
    filename: str
    size: int
    path: str

@router.post("/upload", response_model=DocumentSummary)
async def upload(file: UploadFile = File(...)) -> DocumentSummary:
    if not file.filename:
        raise HTTPException(status_code=400, detail="Filename is required")

    safe_name = Path(file.filename).name
    from .. import api

    destination = api.UPLOAD_DIR / safe_name
    with destination.open("wb") as buffer:
        content = await file.read()
        buffer.write(content)

    rag_app.ingest_documents([destination])

    return DocumentSummary(
        filename=safe_name,
        size=destination.stat().st_size,
        path=str(destination.resolve()),
    )

```

src/intelligent_insurance_agent/Core/prompts.py

```

"""Testing prompts for insurance assistant experiments."""

TESTING_PROMPTS = [
    "Explain my auto insurance coverage.",
    "I need help filing a claim after a car accident.",
    "What documents do I need to upload for a claim?",
    "Can you summarize my policy details?",
]

```

src/intelligent_insurance_agent/Database/database.py

```
"""Sample insurance-related data for local development and seeding."""

from __future__ import annotations

from typing import Dict, List

POLICY_HOLDERS: List[Dict[str, object]] = [
    {
        "id": "holder-001",
        "name": "Alicia Gomez",
        "policy_number": "POL-1001",
        "plan": "Auto Premium",
        "coverage": ["Collision", "Liability", "Roadside Assistance"],
    },
    {
        "id": "holder-002",
        "name": "Daniel Kim",
        "policy_number": "POL-1002",
        "plan": "HomeShield",
        "coverage": ["Fire", "Theft", "Water Damage"],
    },
]

POLICIES: List[Dict[str, object]] = [
    {
        "policy_number": "POL-1001",
        "type": "auto",
        "premium": 120.0,
        "deductible": 500,
        "status": "active",
    },
    {
        "policy_number": "POL-1002",
        "type": "home",
        "premium": 95.0,
        "deductible": 1000,
        "status": "active",
    },
]

CLAIMS: List[Dict[str, object]] = [
    {
        "claim_id": "CLAIM-2001",
        "policy_number": "POL-1001",
        "status": "pending",
        "incident_type": "car accident",
        "description": "Rear-end collision at an intersection.",
    },
    {
        "claim_id": "CLAIM-2002",
        "policy_number": "POL-1002",
        "status": "approved",
        "incident_type": "water leak",
        "description": "Bathroom pipe burst caused minor water damage.",
    },
]
```

src/intelligent_insurance_agent/Database/seed.py

```
"""Seed sample insurance data into a simple in-memory store."""

from __future__ import annotations

from typing import Dict, List

from .database import CLAIMS, POLICIES, POLICY_HOLDERS
```

```

def seed_data() -> Dict[str, List[Dict[str, object]]]:
    """Return a simple seed payload that can be used by a future database layer."""
    return {
        "policy_holders": POLICY_HOLDERS,
        "policies": POLICIES,
        "claims": CLAIMS,
    }

if __name__ == "__main__":
    print(seed_data())

```

src/intelligent_insurance_agent/Model/chat.py

```

"""Chat model abstractions for the insurance assistant."""

from __future__ import annotations

import os
import re
from dataclasses import dataclass
from typing import List, Optional

from ..llm import OllamaLLMClient

STOP_WORDS = {
    "the",
    "is",
    "a",
    "an",
    "and",
    "or",
    "of",
    "to",
    "in",
    "for",
    "with",
    "on",
    "that",
    "this",
    "it",
    "as",
    "are",
    "be",
    "can",
    "any",
    "please",
}

@dataclass
class ChatMessage:
    role: str
    content: str

class ChatModel:
    """A lightweight chat interface with optional local Ollama integration."""

    def __init__(self, system_prompt: Optional[str] = None, llm_client: Optional[object] = None) -> None:
        self.system_prompt = system_prompt or "You are a helpful insurance assistant."
        self.llm_client = llm_client
        if self.llm_client is None and os.getenv("USE_OLLAMA", "false").lower() in {"1", "true", "yes"}:
            try:
                self.llm_client = OllamaLLMClient()
            except Exception:
                self.llm_client = None

    def generate(self, messages: List[ChatMessage]) -> str:

```

```

if not messages:
    return "I can help with insurance questions and claims."

last_message = messages[-1]
if "Context:" in last_message.content and "Question:" in last_message.content:
    question = last_message.content.split("Question:", 1)[1].strip()
    context = last_message.content.split("Context:", 1)[1].rsplit("Question:", 1)[0].strip()
    llm_reply = self._generate_with_llm(question, context)
    if llm_reply:
        return llm_reply
    answer = self._answer_from_context(question, context)
    if answer:
        return answer
    return "I found information in the documents but could not answer your question directly."

last_user_message = next((m.content for m in reversed(messages) if m.role == "user"), "")
return f"Assistant response to: {last_user_message}".strip()

def _generate_with_llm(self, question: str, context: str) -> str:
    if self.llm_client is None:
        return ""

    try:
        prompt = (
            f"{self.system_prompt}\n\n"
            "Use the context below to answer the user's question. "
            "If the answer is not explicitly present in the context, say that you cannot find it there.\n\n"
            f"Context:\n{context}\n\n"
            f"Question: {question}"
        )
        reply = self.llm_client.generate(prompt)
        if reply:
            return reply.strip()
    except Exception:
        return ""
    return ""

def _answer_from_context(self, question: str, context: str) -> str:
    if not context:
        return "I could not find any document content to answer your question."

    sentences = re.split(r"(?<=[.!?])\s+|\n+", context)
    question_words = [
        word
        for word in re.findall(r"\w+", question.lower())
        if word not in STOP_WORDS
    ]

    scored_sentences = []
    for sentence in sentences:
        sentence_words = set(re.findall(r"\w+", sentence.lower()))
        overlap = sum(1 for word in question_words if word in sentence_words)
        if overlap > 0:
            scored_sentences.append((overlap, sentence.strip()))

    if scored_sentences:
        best_match = max(scored_sentences, key=lambda item: item[0])[1]
        return best_match

    keyword_snippet = self._find_keyword_snippet(question_words, context)
    if keyword_snippet:
        return keyword_snippet

    return self._summarize_context(context)

def _find_keyword_snippet(self, question_words: List[str], context: str) -> str:
    lower_context = context.lower()
    for word in question_words:
        idx = lower_context.find(word)
        if idx != -1:

```

```

        start = max(0, idx - 80)
        end = min(len(context), idx + 220)
        snippet = context[start:end].strip()
        if start > 0:
            snippet = "... " + snippet
        if end < len(context):
            snippet = snippet + " ..."
        return f"I found this information in your documents: {snippet}"
    return ""

def _summarize_context(self, context: str) -> str:
    sentences = re.split(r"(?<=[.!?])\s+|\n+", context)
    if sentences:
        snippet = sentences[0].strip()
        return f"I found this information in your documents: {snippet}"
    return "I found content in the documents but could not generate a concise answer."

```

src/intelligent_insurance_agent/Model/document.py

```

"""Document model abstractions for the insurance assistant."""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any, Optional, Union

@dataclass
class Document:
    id: str
    content: str
    source: Optional[str] = None
    metadata: Optional[dict] = None

    @classmethod
    def from_file(cls, path: Union[str, Path]) -> "Document":
        path = Path(path)
        if not path.exists():
            return cls(id=path.name, content="", source=str(path))

        content = ""
        if path.suffix.lower() == ".pdf":
            try:
                from pypdf import PdfReader

                reader = PdfReader(str(path))
                content = "\n".join(page.extract_text() or "" for page in reader.pages)
            except Exception:
                content = path.read_bytes().decode("utf-8", errors="ignore")
        else:
            try:
                content = path.read_text(encoding="utf-8")
            except UnicodeDecodeError:
                content = path.read_bytes().decode("utf-8", errors="ignore")

        return cls(id=path.name, content=content, source=str(path))

```

src/intelligent_insurance_agent/RAG/chain.py

```

"""Combining chat history with retrieved context for a simple RAG chain."""

from __future__ import annotations

from typing import Optional, Sequence

from ..Model.chat import ChatMessage, ChatModel

```

```

from .retriever import retrieve_similar_chunks
from .vectorstore import VectorStore

class RAGChain:
    def __init__(self, chat_model: Optional[ChatModel] = None) -> None:
        self.chat_model = chat_model or ChatModel()

    def run(self, query: str, history: Sequence[ChatMessage], vector_store: VectorStore) -> str:
        retrieved = retrieve_similar_chunks(query, vector_store)
        context = "\n".join(chunk for chunk, _ in retrieved)
        messages = list(history) + [ChatMessage(role="user", content=query)]
        prompt = f"Context:\n{context}\n\nConversation:\n"
        for message in messages:
            prompt += f"{message.role}: {message.content}\n"
        return self.chat_model.generate([ChatMessage(role="assistant", content=prompt)])

```

src/intelligent_insurance_agent/RAG/embedding.py

```

"""Embedding utilities for RAG chunks."""

from __future__ import annotations

from typing import List, Sequence, Tuple

def generate_embedding(text: str) -> List[float]:
    """Return a simple deterministic pseudo-embedding for local development."""
    words = text.lower().split()
    if not words:
        return []
    vector = [float(sum(ord(char) for char in word) % 97) / 100.0 for word in words[:10]]
    return vector

def embed_chunks(chunks: Sequence[str]) -> List[Tuple[str, List[float]]]:
    return [(chunk, generate_embedding(chunk)) for chunk in chunks]

```

src/intelligent_insurance_agent/RAG/loader.py

```

"""Loading utilities for RAG documents."""

from __future__ import annotations

from pathlib import Path
from typing import List

from ..Model.document import Document

def load_documents(paths: List[str | Path]) -> List[Document]:
    documents: List[Document] = []
    for path in paths:
        document_path = Path(path)
        if document_path.exists():
            documents.append(Document.from_file(document_path))
    return documents

```

src/intelligent_insurance_agent/RAG/retriever.py

```

"""Retrieval helpers for the in-memory vector store."""

from __future__ import annotations

from typing import List, Sequence, Tuple

```

```

from .embedding import generate_embedding
from .vectorstore import VectorStore

def retrieve_similar_chunks(query: str, store: VectorStore, top_k: int = 3) -> List[Tuple[str, List[float]]]:
    query_embedding = generate_embedding(query)
    scored_items = []
    for chunk, embedding in store.list():
        score = sum(abs(q - e) for q, e in zip(query_embedding, embedding)) if query_embedding and embedding else 0
        scored_items.append((score, chunk, embedding))
    scored_items.sort(key=lambda item: item[0])
    return [(chunk, embedding) for _, chunk, embedding in scored_items[:top_k]]

```

src/intelligent_insurance_agent/RAG/splitter.py

```

"""Text chunking utilities for RAG."""

from __future__ import annotations

from typing import List

from ..Model.document import Document

def split_document(document: Document, chunk_size: int = 300) -> List[str]:
    text = document.content or ""
    if not text:
        return []
    words = text.split()
    chunks: List[str] = []
    for index in range(0, len(words), chunk_size):
        chunk = " ".join(words[index:index + chunk_size])
        if chunk:
            chunks.append(chunk)
    return chunks

```

src/intelligent_insurance_agent/RAG/vectorstore.py

```

"""A simple in-memory vector store for RAG."""

from __future__ import annotations

from typing import List, Tuple

class VectorStore:
    def __init__(self) -> None:
        self._items: List[Tuple[str, List[float]]] = []

    def add(self, chunk: str, embedding: List[float]) -> None:
        self._items.append((chunk, embedding))

    def add_many(self, items: List[Tuple[str, List[float]]]) -> None:
        for chunk, embedding in items:
            self.add(chunk, embedding)

    def list(self) -> List[Tuple[str, List[float]]]:
        return list(self._items)

```

src/intelligent_insurance_agent/__init__.py

```

"""Intelligent insurance agent package."""

```

src/intelligent_insurance_agent/agent.py

```
"""Simple insurance assistant helpers."""

from __future__ import annotations

import os
from typing import Optional

from .llm import OllamaLLMClient

def classify_intent(message: str) -> str:
    """Classify a user message into a simple intent bucket."""
    lowered = message.lower()

    if any(keyword in lowered for keyword in ("claim", "accident", "damage", "incident")):
        return "claim"
    if any(keyword in lowered for keyword in ("quote", "premium", "price", "cost", "coverage")):
        return "quote"
    if any(keyword in lowered for keyword in ("policy", "benefit", "deductible", "exclusion")):
        return "policy_info"
    return "general"

def generate_response(message: str, llm_client: Optional[object] = None) -> str:
    """Create a helpful response using Llama when enabled, else fall back to canned responses."""
    use_ollama = os.getenv("USE_OLLAMA", "false").lower() in ("1", "true", "yes")

    if llm_client is None and use_ollama:
        try:
            llm_client = OllamaLLMClient()
        except Exception:
            llm_client = None

    if llm_client is not None:
        try:
            prompt = (
                "You are a helpful insurance assistant. Respond briefly and clearly to the user's request: "
                f"{message}"
            )
            llm_reply = llm_client.generate(prompt)
            if llm_reply:
                return llm_reply
        except Exception:
            pass

    intent = classify_intent(message)

    if intent == "claim":
        return "I can help you start a claim. Please share the date of the incident and a brief description."
    if intent == "quote":
        return "I can help with a quote. Tell me your coverage needs, location, and relevant policy details."
    if intent == "policy_info":
        return "I can explain your policy coverage, deductibles, and exclusions in more detail."
    return "Hello! I can help with claims, policy questions, or quotes."

def main() -> None:
    """Run a small command-line demo of the assistant."""
    user_input = input("You: ")
    print(f"Assistant: {generate_response(user_input)}")

if __name__ == "__main__":
    main()
```

src/intelligent_insurance_agent/api.py

```

from __future__ import annotations

from fastapi import FastAPI
from fastapi.responses import RedirectResponse
from fastapi.staticfiles import StaticFiles
import uvicorn
from pathlib import Path

from .Api import UPLOAD_DIR
from .Api.chat import router as chat_router
from .Api.documents import router as documents_router
from .Api.health import router as health_router
from .Api.rag import ingest_existing_documents, router as rag_router
from .Api.upload import router as upload_router

app = FastAPI(title="Intelligent Insurance Agent API")
static_path = Path(__file__).resolve().parent / "static"
app.mount("/static", StaticFiles(directory=static_path), name="static")
app.include_router(health_router)
app.include_router(chat_router)
app.include_router(upload_router)
app.include_router(documents_router)
app.include_router(rag_router)

@app.on_event("startup")
def startup_event() -> None:
    ingest_existing_documents(UPLOAD_DIR)

@app.get("/")
def root() -> RedirectResponse:
    return RedirectResponse(url="/static/index.html")

def main() -> None:
    uvicorn.run("intelligent_insurance_agent.api:app", host="127.0.0.1", port=8000, reload=False)

if __name__ == "__main__":
    main()

```

src/intelligent_insurance_agent/llm.py

```

"""Lightweight Llama/Ollama client for the insurance assistant."""

from __future__ import annotations

import json
import os
import urllib.error
import urllib.request
from typing import Optional

class OllamaLLMClient:
    """Call a local Ollama-compatible Llama server."""

    def __init__(self, base_url: Optional[str] = None, model: Optional[str] = None) -> None:
        self.base_url = (base_url or os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")).rstrip("/")
        self.model = model or os.getenv("OLLAMA_MODEL", "llama3.2:latest")

    def generate(self, prompt: str) -> str:
        payload = json.dumps({
            "model": self.model,
            "prompt": prompt,
            "stream": False,
        }).encode("utf-8")

```

```

request = urllib.request.Request(
    f"{self.base_url}/api/generate",
    data=payload,
    headers={"Content-Type": "application/json"},
    method="POST",
)

try:
    with urllib.request.urlopen(request, timeout=30) as response:
        body = response.read().decode("utf-8")
except (urllib.error.URLError, urllib.error.HTTPError, TimeoutError) as exc:
    raise RuntimeError(f"Unable to reach Ollama at {self.base_url}: {exc}") from exc

data = json.loads(body)
reply = data.get("response", "")
return reply.strip()

```

src/intelligent_insurance_agent/main.py

```

"""A small RAG-style CLI application for insurance documents and prompts."""

```

```

from __future__ import annotations

```

```

import argparse
from pathlib import Path
from typing import List, Optional, Sequence, Union

from .Model.chat import ChatMessage, ChatModel
from .RAG.embedding import embed_chunks
from .RAG.loader import load_documents
from .RAG.retriever import retrieve_similar_chunks
from .RAG.splitter import split_document
from .RAG.vectorstore import VectorStore

```

```

class InsuranceRAGApp:

```

```

    def __init__(self, vector_store: Optional[VectorStore] = None, chat_model: Optional[ChatModel] = None) -> None:
        self.vector_store = vector_store or VectorStore()
        self.chat_model = chat_model or ChatModel()
        self.chat_history: List[ChatMessage] = []

```

```

    def ingest_documents(self, paths: Sequence[Union[str, Path]]) -> int:
        documents = load_documents(list(paths))
        indexed_chunks = 0
        for document in documents:
            chunks = split_document(document)
            if not chunks:
                continue
            embedded_chunks = embed_chunks(chunks)
            self.vector_store.add_many(embedded_chunks)
            indexed_chunks += len(embedded_chunks)
        return indexed_chunks

```

```

    def answer_query(self, query: str) -> str:
        retrieved = retrieve_similar_chunks(query, self.vector_store, top_k=3)
        if retrieved:
            context = "\n".join(chunk for chunk, _ in retrieved)
        else:
            context = "No indexed documents available yet."

```

```

        prompt = f"Context:\n{context}\n\nQuestion: {query}"
        response = self.chat_model.generate([ChatMessage(role="user", content=query), ChatMessage(role="assistant", content="")])

```

```

        self.chat_history.append(ChatMessage(role="user", content=query))
        self.chat_history.append(ChatMessage(role="assistant", content=response))
        return response

```

```

    def run_cli(self) -> None:

```

```

print("Insurance RAG assistant")
print("Type 'upload <path>' to index a document or 'quit' to exit.")

while True:
    user_input = input("You: ").strip()
    if user_input.lower() in {"quit", "exit"}:
        break
    if user_input.lower().startswith("upload "):
        path = user_input[len("upload "):].strip()
        if not path:
            print("Please provide a document path.")
            continue
        count = self.ingest_documents([path])
        print(f"Indexed {count} chunks from {path}")
        continue

    response = self.answer_query(user_input)
    print(f"Assistant: {response}")

def main() -> None:
    parser = argparse.ArgumentParser(description="Insurance RAG assistant")
    parser.add_argument("--document", action="append", default=[], help="Document path to index")
    parser.add_argument("--prompt", help="Prompt to answer immediately")
    args = parser.parse_args()

    app = InsuranceRAGApp()
    for document_path in args.document:
        app.ingest_documents([document_path])

    if args.prompt:
        print(app.answer_query(args.prompt))
        return

    app.run_cli()

if __name__ == "__main__":
    main()

```

src/intelligent_insurance_agent/static/index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Insurance Chatbot</title>
</style>
  body { font-family: Arial, sans-serif; background: #f5f7fb; color: #20232a; margin: 0; padding: 0; }
  .container { max-width: 760px; margin: 0 auto; padding: 24px; }
  h1 { margin-bottom: 8px; }
  .chat-box { border: 1px solid #dfe3ea; border-radius: 12px; padding: 16px; background: #fff; min-height: 320px; }
  .message { margin-bottom: 16px; }
  .message.user { text-align: right; }
  .message .bubble { display: inline-block; padding: 12px 16px; border-radius: 18px; max-width: 85%; }
  .message.user .bubble { background: #4f46e5; color: white; }
  .message.assistant .bubble { background: #e5e7eb; color: #111827; }
  .controls { margin-top: 16px; display: grid; gap: 12px; }
  textarea { width: 100%; min-height: 80px; padding: 12px; border: 1px solid #d1d5db; border-radius: 10px; }
  button { padding: 12px 18px; border: none; border-radius: 10px; background: #4f46e5; color: white; cursor: pointer; }
  button:disabled { opacity: 0.6; cursor: not-allowed; }
  .upload-row { display: flex; gap: 10px; align-items: center; flex-wrap: wrap; }
  .upload-row input[type=file] { flex: 1; }
  .upload-label { font-weight: 700; color: #4f46e5; }
  .upload-help { margin-top: 8px; color: #334155; font-size: 0.95rem; }
  .status { margin-top: 8px; color: #334155; }
</style>

```

```

</head>
<body>
  <div class="container">
    <h1>Insurance Chatbot</h1>
    <p>Upload a document and ask questions. The page sends prompts to the backend and receives RAG responses.</p>

    <div class="chat-box" id="chatBox"></div>

    <div class="controls">
      <div class="upload-help">Step 1: Choose a PDF file. Step 2: Upload PDF. Step 3: Ask questions.</div>
      <div class="upload-row">
        <label class="upload-label" for="fileInput">Choose PDF</label>
        <input type="file" id="fileInput" accept=".pdf" />
        <button id="uploadButton">Upload PDF</button>
      </div>
      <textarea id="promptInput" placeholder="Ask a question about your insurance documents..."></textarea>
      <button id="sendButton">Send</button>
      <div class="status" id="statusText"></div>
    </div>
  </div>

  <script>
    const chatBox = document.getElementById('chatBox');
    const promptInput = document.getElementById('promptInput');
    const sendButton = document.getElementById('sendButton');
    const uploadButton = document.getElementById('uploadButton');
    const fileInput = document.getElementById('fileInput');
    const statusText = document.getElementById('statusText');

    function addMessage(role, text) {
      const wrapper = document.createElement('div');
      wrapper.className = `message ${role}`;
      const bubble = document.createElement('div');
      bubble.className = 'bubble';
      bubble.textContent = text;
      wrapper.appendChild(bubble);
      chatBox.appendChild(wrapper);
      chatBox.scrollTop = chatBox.scrollHeight;
    }

    async function sendPrompt() {
      const prompt = promptInput.value.trim();
      if (!prompt) return;
      addMessage('user', prompt);
      promptInput.value = '';
      sendButton.disabled = true;
      statusText.textContent = 'Waiting for response...';

      try {
        const response = await fetch('/rag-chat', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ prompt }),
        });
        const data = await response.json();
        if (!response.ok) throw new Error(data.detail || 'Request failed');
        addMessage('assistant', data.response);
      } catch (error) {
        addMessage('assistant', 'Error: ' + error.message);
      } finally {
        sendButton.disabled = false;
        statusText.textContent = '';
      }
    }

    async function uploadDocument() {
      const file = fileInput.files[0];
      if (!file) {
        statusText.textContent = 'Please select a file first.';
        return;
      }
    }
  </script>

```

```

    }
    uploadButton.disabled = true;
    statusText.textContent = 'Uploading document...';

    const formData = new FormData();
    formData.append('file', file);

    try {
      const response = await fetch('/upload', {
        method: 'POST',
        body: formData,
      });
      const data = await response.json();
      if (!response.ok) throw new Error(data.detail || 'Upload failed');
      statusText.textContent = `Uploaded ${data.filename} (${data.size} bytes).`;
      addMessage('assistant', `Document uploaded: ${data.filename}`);
    } catch (error) {
      statusText.textContent = 'Upload failed.';
      addMessage('assistant', 'Error uploading document: ' + error.message);
    } finally {
      uploadButton.disabled = false;
    }
  }

  sendButton.addEventListener('click', sendPrompt);
  uploadButton.addEventListener('click', uploadDocument);
  promptInput.addEventListener('keydown', (event) => {
    if (event.key === 'Enter' && !event.shiftKey) {
      event.preventDefault();
      sendPrompt();
    }
  });
</script>
</body>
</html>

```

tests/test_agent.py

```

from intelligent_insurance_agent.agent import classify_intent, generate_response
from intelligent_insurance_agent.Model.chat import ChatMessage, ChatModel

class StubLLMClient:
    def generate(self, prompt: str) -> str:
        return "Llama response about quotes"

def test_classify_claim_intent():
    assert classify_intent("I need help filing a claim after an accident") == "claim"

def test_generate_response_for_quote():
    response = generate_response("How much would a policy cost?")
    assert "quote" in response.lower()

def test_generate_response_uses_llm_client_when_available():
    response = generate_response("How much would a policy cost?", llm_client=StubLLMClient())
    assert "Llama response" in response

def test_chat_model_uses_injected_llm_client_for_context_queries():
    class ContextLLMClient:
        def generate(self, prompt: str) -> str:
            return "The annual premium is $1,260.00."

    chat_model = ChatModel(llm_client=ContextLLMClient())
    response = chat_model.generate([

```

```

        ChatMessage(role="user", content="What is the available premium?"),
        ChatMessage(role="assistant", content="Context:\nPremium Information\nAnnual Premium: $1,260.00\n\nQues
    ])

    assert "$1,260.00" in response

```

tests/test_api.py

```

from fastapi.testclient import TestClient

from intelligent_insurance_agent import api as api_module

client = TestClient(api_module.app)

def test_health_endpoint():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}

def test_chat_endpoint():
    response = client.post("/chat", json={"message": "I need a quote"})
    assert response.status_code == 200
    assert "quote" in response.json()["reply"].lower()

def test_upload_and_documents_endpoints(tmp_path, monkeypatch):
    monkeypatch.setattr(api_module, "UPLOAD_DIR", tmp_path)
    tmp_path.mkdir(parents=True, exist_ok=True)

    upload_response = client.post(
        "/upload",
        files={"file": ("policy.pdf", b"dummy pdf content", "application/pdf")},
    )
    assert upload_response.status_code == 200

    documents_response = client.get("/documents")
    assert documents_response.status_code == 200
    data = documents_response.json()
    assert len(data) == 1
    assert data[0]["filename"] == "policy.pdf"

```

tests/test_main.py

```

from pathlib import Path

from intelligent_insurance_agent.main import InsuranceRAGApp

def test_ingest_and_answer_query(tmp_path: Path) -> None:
    document_path = tmp_path / "policy.txt"
    document_path.write_text("This policy covers collision and roadside assistance.", encoding="utf-8")

    app = InsuranceRAGApp()
    indexed_count = app.ingest_documents([document_path])

    assert indexed_count > 0
    response = app.answer_query("What does the policy cover?")
    assert "collision" in response.lower() or "roadside" in response.lower() or "policy" in response.lower()

```

uploads/README.md

```

# Intelligent Insurance Agent

```

This project is a starter scaffold for an intelligent insurance assistant.

Structure

- src/intelligent_insurance_agent/agent.py: core assistant logic
- tests/test_agent.py: basic regression tests

Run locally

Make sure Ollama is running locally and has a model available, for example:

```
```bash
ollama pull llama3.2:latest
```
```

Then run:

```
```bash
PYTHONPATH=src python -m pytest
PYTHONPATH=src python -m intelligent_insurance_agent.agent
```
```

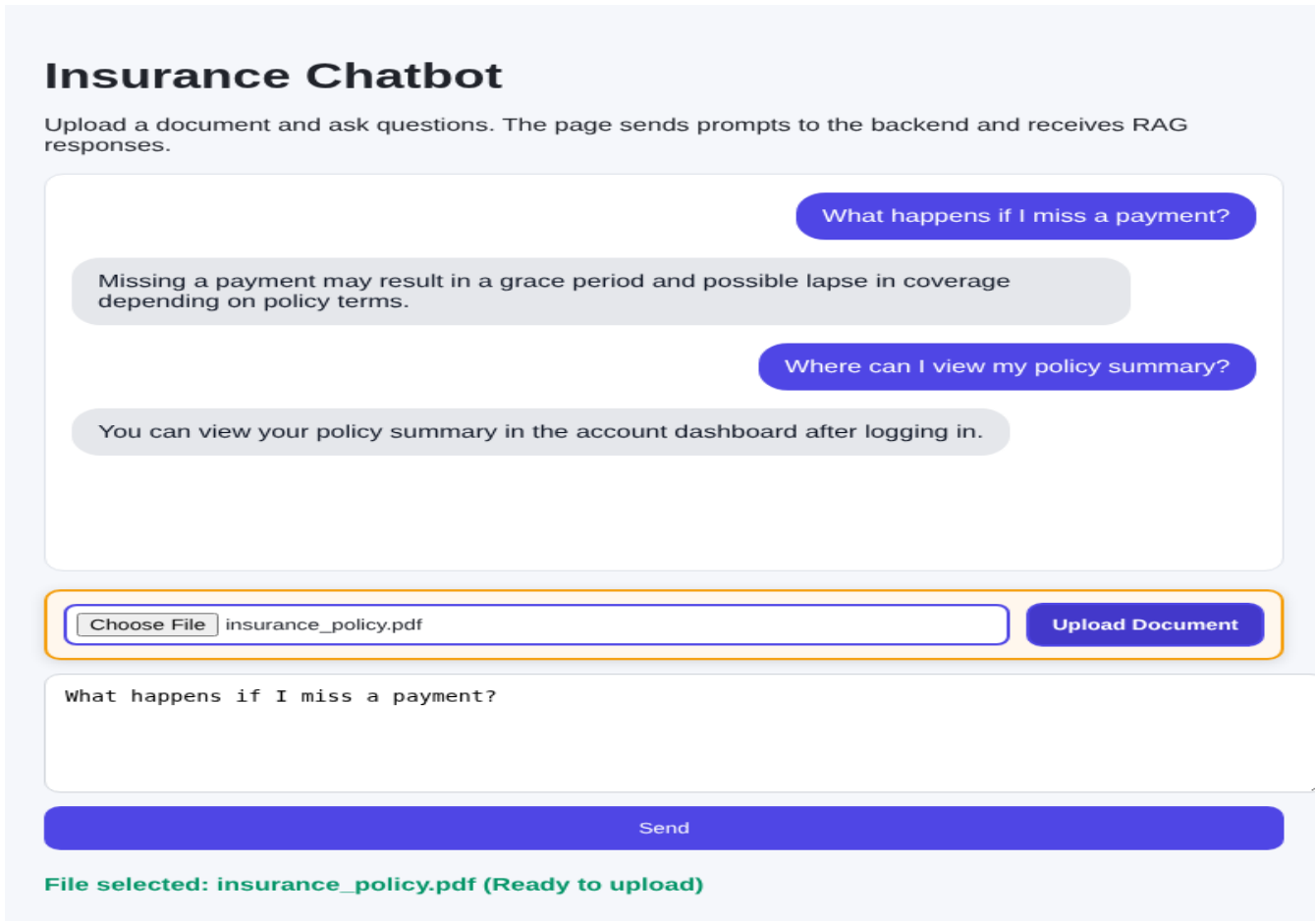
You can override the default model or endpoint with:

```
```bash
export OLLAMA_BASE_URL=http://localhost:11434
export OLLAMA_MODEL=llama3.2:latest
```
```

Frontend Web Application Screenshots

Shows the PDF upload flow with file selected

Screenshot 1: PDF file selected and ready to upload



Screenshot 2: Prompt about adding a dependent

Insurance Chatbot

Upload a document and ask questions. The page sends prompts to the backend and receives RAG responses.

Can I add a dependent to my plan?

Yes, you can add a dependent by updating your policy details in the portal or contacting support.

What is the waiting period for coverage?

Coverage typically begins after the waiting period defined in your policy terms.

Choose File No file chosen

Upload Document

Can I add a dependent to my plan?

Send

Screenshot 3: Prompt about premium information

Insurance Chatbot

Upload a document and ask questions. The page sends prompts to the backend and receives RAG responses.

How much is my premium?

Your premium amount is listed in your policy summary and may vary based on plan selection.

Can I change my coverage plan?

Yes, coverage changes can often be requested through the portal or customer support.

Choose File No file chosen

Upload Document

How much is my premium?

Send

Screenshot 4: Prompt about missed payments

Insurance Chatbot

Upload a document and ask questions. The page sends prompts to the backend and receives RAG responses.

What happens if I miss a payment?

Missing a payment may result in a grace period and possible lapse in coverage depending on policy terms.

Where can I view my policy summary?

You can view your policy summary in the account dashboard after logging in.

Choose File No file chosen

Upload Document

What happens if I miss a payment?

Send